

DEVARA HEPSIBA

2403A51L25

BATCH-51

Lab 13: Code Refactoring – Improving Legacy Code with AI

Suggestions

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

Prompt: Refactor the given Python program by identifying repeated calculations and converting them into a reusable function. Ensure the program produces the same output and include proper documentation for the function.

Sample Code:

```
length = 8
width = 4

print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6

print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Code:

```
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        tuple: A tuple containing (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

# First rectangle
length = 8
width = 4
area, perimeter = calculate_rectangle_properties(length, width)
print("Area:", area)
print("Perimeter:", perimeter)

# Second rectangle
length = 12
width = 6
area, perimeter = calculate_rectangle_properties(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

Output:

```
Area: 32  
Perimeter: 24  
Area: 72  
Perimeter: 36
```

Explanation: The repeated calculations for area and perimeter are moved into a reusable function. This reduces code duplication and improves maintainability. The program output remains the same while making the code cleaner.

Task Description -2 (Refactoring – Modularizing Inline Calculations)

Prompt: The Python script and extract repeated inline calculations into a reusable function. Improve code readability and maintain proper documentation while maintaining the same output.

Sample Code:

```
amount = 1000  
  
discount = amount * 0.10  
  
final_amount = amount - discount  
print("Final Amount:", final_amount)  
  
amount = 2000  
  
discount = amount * 0.10  
  
final_amount = amount - discount  
print("Final Amount:", final_amount)
```

Code:

```
def calculate_discounted_amount(amount, discount_rate=0.10):
    """
    Calculate the final amount after applying a discount.

    Args:
        amount (float): The original amount before discount
        discount_rate (float): The discount rate as a decimal (default: 0.10 for 10%)

    Returns:
        dict: A dictionary containing the original amount, discount, and final amount
    """
    discount = amount * discount_rate
    final_amount = amount - discount

    return {
        "original_amount": amount,
        "discount": discount,
        "final_amount": final_amount
    }

# Test with different amounts
amounts = [1000, 2000]

for amount in amounts:
    result = calculate_discounted_amount(amount)
    print(f"Original Amount: ${result['original_amount']}")
    print(f"Discount: ${result['discount']:.2f}")
    print(f"Final Amount: ${result['final_amount']:.2f}")
    print()
```

Output:

```
Original Amount: $1000
Discount: $100.00
Final Amount: $900.00

Original Amount: $2000
Discount: $200.00
Final Amount: $1800.00
```

Explanation: The repeated discount calculation is converted into a reusable function. This improves modularity and code reuse. It also makes the program easier to maintain.

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

Prompt: Optimize the given Python program by reducing inefficient nested loops and improving lookup performance using suitable data structures. Ensure the logic and output remain unchanged.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
```

```
check_list = ["Charan", "Esha", "Bhavya"]
```

```
for name in check_list:
```

```
exists = False
```

```
for student in students:

if name == student:

exists = True

if exists:

print(name, "found")

else:

print(name, "not found")
```

Code:

```
import time

# Original inefficient code
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

print("=== Original Code (Nested Loops) ===")
start = time.time()
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
original_time = time.time() - start
print(f"Execution time: {original_time:.6f} seconds\n")

# Optimized version 1: Using 'in' operator with list
print("=== Optimized Code 1 (Using 'in' operator) ===")
start = time.time()
for name in check_list:
    if name in students:
        print(name, "found")
    else:
        print(name, "not found")
optimized_time_1 = time.time() - start
print(f"Execution time: {optimized_time_1:.6f} seconds\n")

# Optimized version 2: Using set for O(1) lookup
print("=== Optimized Code 2 (Using Set) ===")
start = time.time()
students_set = set(students)
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")
optimized_time_2 = time.time() - start
print(f"Execution time: {optimized_time_2:.6f} seconds\n")

# Performance comparison
print("=== Performance Summary ===")
print(f"Original (nested loops): {original_time:.6f}s")
print(f"Optimized 1 ('in' operator): {optimized_time_1:.6f}s")
print(f"Optimized 2 (set lookup): {optimized_time_2:.6f}s")
print(f"Speedup (set vs original): {original_time/optimized_time_2:.2f}x faster")
```

Output:

```
=== Original Code (Nested Loops) ===
Charan found
Esha not found
Bhavya found
Execution time: 0.000640 seconds

=== Optimized Code 1 (Using 'in' operator) ===
Charan found
Esha not found
Bhavya found
Execution time: 0.000180 seconds

=== Optimized Code 2 (Using Set) ===
Charan found
Esha not found
Bhavya found
Execution time: 0.000244 seconds

=== Performance Summary ===
Original (nested loops): 0.000640s
Optimized 1 ('in' operator): 0.000180s
Optimized 2 (set lookup): 0.000244s
Speedup (set vs original): 2.62x faster
D:\Users\hansh\OneDrive\Desktop>
```

Explanation: Nested loops are replaced with a **set for faster lookup**.

Sets provide **O (1) lookup time**, improving performance.

This reduces unnecessary comparisons.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

prompt: Refactor the program by replacing hardcoded numeric values with meaningful constants defined at the beginning of the script. Ensure calculations remain correct and the code becomes easier to maintain.

Sample Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

code:

```
# Define constants at the beginning
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 2
TIME = 5
DIVISOR = 100

# Calculate simple interest using constants
print("Simple Interest:", (PRINCIPAL_1 * RATE * TIME) / DIVISOR)
print("Simple Interest:", (PRINCIPAL_2 * RATE * TIME) / DIVISOR)
```

output:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

Explanation: Hardcoded numbers are replaced with constants like RATE and TIME. This improves code readability and maintainability. Changing values becomes easier without modifying calculations.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

prompt: Improve the readability of the given Python code by renaming unclear variables with meaningful names and adding comments to explain the logic without changing the program output.

Sample Code:

```
x = 50
y = 30
z = x + y
print(z)
```

code:

```
# Initialize the first number
first_number = 50

# Initialize the second number
second_number = 30

# Calculate the sum of both numbers
total_sum = first_number + second_number

# Display the result
print(total_sum)
```

Output:

```
80
PS C:\Users\hepsi\OneDrive\Desktop\AI>
```

Explanation: Unclear variable names like x, y, z are replaced with meaningful names. Comments are added to explain the logic. This makes the code easier to understand.